# Aquarium Tank REST API

## API tutorial

Daniel Than · ASE 230 · Project 1

PHP, PDO, and MySQL

10 endpoints for aquarium tanks, water readings, and plants.

# What the API does

- Store a tank's name and size in US gallons.
- Record pH and temperature in Celsius for each tank.
- Keep a list of plants in each tank.
- Return JSON for use by a browser or cURL.

Three operations require a Bearer token.

# How a request works

1. The browser or cURL sends an HTTP method and path.
2. index.php chooses the route and checks authentication.
3. PHP validates the request and runs prepared SQL through PDO.
4. MySQL stores or returns the records.
5. PHP sends an HTTP status and a JSON response.

# Three related tables

| Table | Main fields |
|---|---|
| tanks | id, name, gallons |
| readings | id, tank_id, ph, temperature_c, recorded_at |
| plants | id, tank_id, name |

readings.tank_id and plants.tank_id reference tanks.id.
Deleting a tank deletes its readings and plants.

Source: MySQL 8.4 manual, FOREIGN KEY Constraints.

# Start the development server

After following the database setup in README.md:

```
export DB_USER=aquarium_project1
export DB_PASSWORD='your-database-password'
export API_TOKEN='your-own-long-token'
php -S localhost:8080 -t code code/router.php
```

Run this from the project root. Stop with Ctrl+C.

Open http://localhost:8080/test_client.html.

# Bearer authentication

Send this header on the three protected endpoints:

```
Authorization: Bearer your-own-long-token
```

- POST /tanks
- DELETE /tanks/{id}
- POST /readings

Missing or incorrect token: HTTP 401.

The server compares the token with API_TOKEN using hash_equals().

# Request examples

The following slides use sample IDs to keep examples readable.

Use the actual ID returned by your POST request.

```
export BASE_URL=http://localhost:8080
export API_TOKEN='your-own-long-token'
```

Numbers in JSON must be numbers, not quoted strings.

The examples show expected response shapes, not fixed database contents.

# GET /tanks

List all tanks. An empty database returns [].

```
curl "$BASE_URL/tanks"
```

HTTP 200

```
[{"id":1,"name":"Living room","gallons":"20.00"}]
```

# GET /tanks/{id}

Get one tank. A missing tank returns 404.

```
curl "$BASE_URL/tanks/1"
```

HTTP 200

```
{"id":1,"name":"Living room","gallons":"20.00"}
```

# POST /tanks

Create a tank. Bearer token required. name and gallons are required.

```
curl -X POST "$BASE_URL/tanks" \
  -H "Authorization: Bearer $API_TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"name":"Living room","gallons":20}'
```

HTTP 201

```
{"id":1,"message":"Tank created"}
```

# PUT /tanks/{id}

Update a tank. Send both name and gallons. This route is public.

```
curl -X PUT "$BASE_URL/tanks/1" \
  -H 'Content-Type: application/json' \
  -d '{"name":"Living room","gallons":25}'
```

HTTP 200

```
{"message":"Tank updated"}
```

# DELETE /tanks/{id}

Delete the tank and all its readings and plants. Bearer token required.

```
curl -X DELETE "$BASE_URL/tanks/1" \
  -H "Authorization: Bearer $API_TOKEN"
```

HTTP 200

```
{"message":"Tank and its readings and plants deleted"}
```

# GET /tanks/{id}/readings

List water readings for an existing tank. New tanks return [].

```
curl "$BASE_URL/tanks/1/readings"
```

HTTP 200

```
[{"id":1,"tank_id":1,"ph":"7.00",
  "temperature_c":"24.00","recorded_at":"2026-10-23 23:00:00"}]
```

# POST /readings

Log a water test. Bearer token required. The tank must already exist.

```
curl -X POST "$BASE_URL/readings" \
  -H "Authorization: Bearer $API_TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"tank_id":1,"ph":7,"temperature_c":24}'
```

HTTP 201

```
{"id":1,"message":"Record created"}
```

# DELETE /readings/{id}

Delete one incorrect or duplicate reading. This route is public.

```
curl -X DELETE "$BASE_URL/readings/1"
```

HTTP 200

```
{"message":"Reading deleted"}
```

# GET /tanks/{id}/plants

List all plants for an existing tank. New tanks return [].

```
curl "$BASE_URL/tanks/1/plants"
```

HTTP 200

```
[{"id":1,"tank_id":1,"name":"Java fern"}]
```

# POST /plants

Add a plant to an existing tank. tank_id and name are required.

```
curl -X POST "$BASE_URL/plants" \
  -H 'Content-Type: application/json' \
  -d '{"tank_id":1,"name":"Java fern"}'
```

HTTP 201

```
{"id":1,"message":"Record created"}
```

# Validation and errors

- IDs must be positive integers.
- name must contain 1 to 100 bytes after trimming.
- gallons must be between 0.01 and 999999.99.
- pH must be between 0 and 14, temperature between 0 and 50 °C.
- tank_id must reference an existing tank.

These input limits are not aquarium care recommendations.

# HTTP status codes

| Status | Meaning |
| --- | --- |
| 200 / 201 | Request succeeded / record created |
| 400 | Invalid JSON, field value, or ID |
| 401 | Missing or incorrect Bearer token |
| 404 | Route or record not found |
| 405 | Method not supported; see Allow header |
| 500 | Server configuration or database failure |

# cURL tests

```
export API_TOKEN='your-own-long-token'
bash code/test_curl.sh
```

The suite creates its own records and uses returned IDs.

It checks all 10 routes, response values, updates, and deletion.

It checks missing/invalid tokens and 400, 404, and 405 errors.

It exits nonzero on failure and attempts cleanup.

# Browser test client

Open http://localhost:8080/test_client.html.

1. Choose an endpoint from the dropdown.
2. Enter the returned ID in the path or JSON body.
3. Enter your token for protected routes.
4. Click Send request and read the HTTP status and JSON.

Test token fields empty, incorrect, and correct.
Serve the page over HTTP so fetch() uses the same origin.

# Security and limits

- Prepared statements bind values separately from SQL.
- hash_equals() compares the configured token.
- Passwords and tokens come from environment variables.
- The router does not serve SQL or PHP source as static text.
- Seven routes are public, as specified in the aquarium plan.
- This is a classroom single-token API; use HTTPS outside localhost.

Sources: PHP PDO::prepare and hash_equals manuals.

# Verified implementation

On October 23, 2026:

- All 10 routes passed against Oracle MySQL 8.4.
- The cURL suite passed on PHP's server and NGINX.
- The browser client passed all routes through NGINX.
- Missing and invalid token requests returned 401.

See presentation/verification.txt and nginx_screenshot.png.

# Sources and project files

- PHP PDO::prepare
- PHP hash_equals
- MySQL foreign keys
- Marp CLI
- Supplied Project 1 overview/rubric and HM4/plan.md.
- HW3 exercise references: REST Q2–Q4 and Security Q3.

Full filenames and question references: README.md.